



Sorayunara (SORA)

Universal Platform Target Matrix & Cross-Compilation Specification v2.0

Desktop & Server (Linux, Windows, macOS)

Mobile (Android, iOS)

Web & Cloud Edge (WASM / WASI)

Embedded & Microcontrollers (ARM Cortex, RISC-V, ESP32)

GPU & Accelerators (CUDA, ROCm, WebGPU)

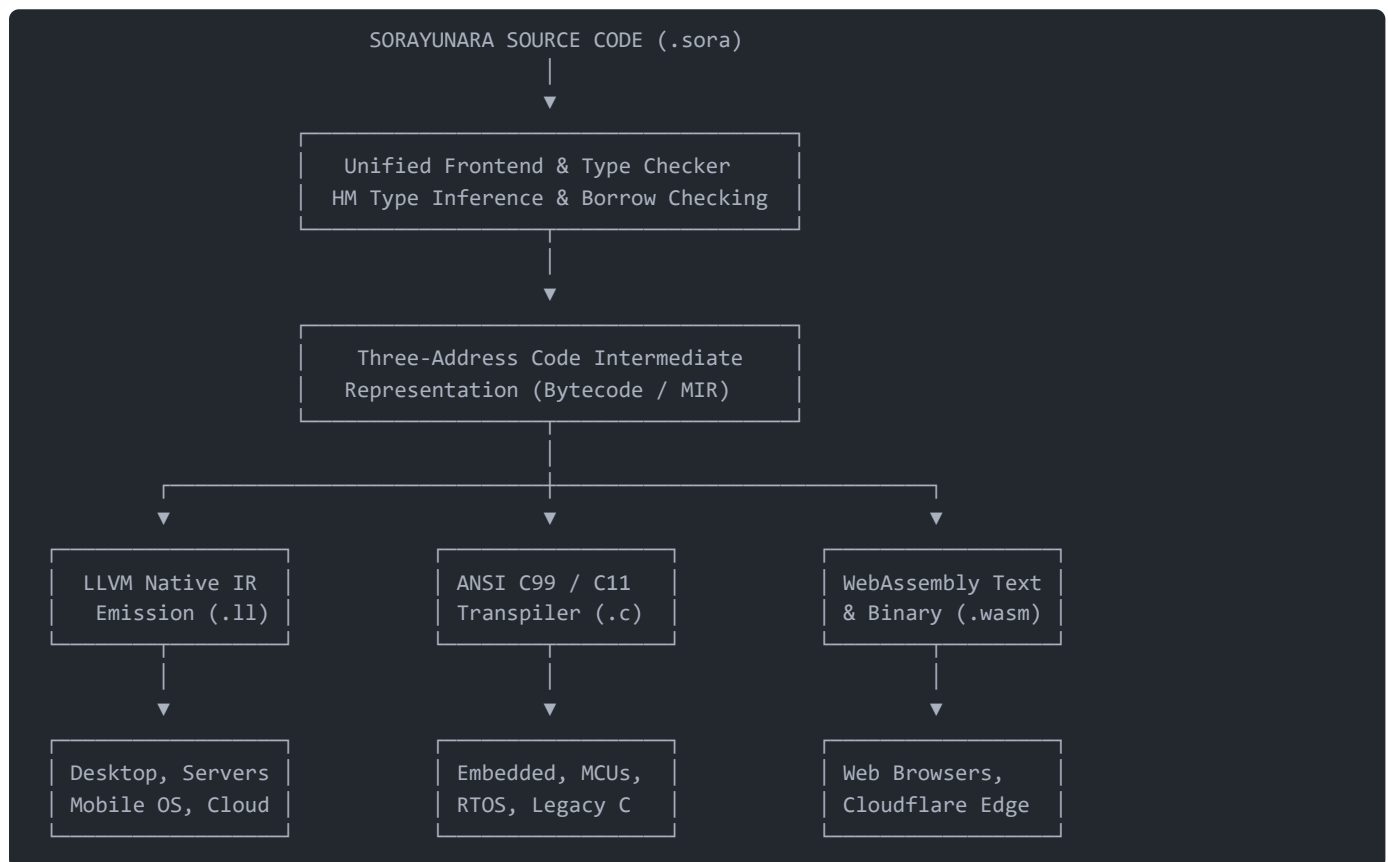
Document ID:	Sorayunara_Sora_Universal_Platform_Target_Matrix.pdf
Compiler Architecture:	Ubiquitous Multi-Target (LLVM / C / WASM / JIT)
Official Repository:	https://github.com/Sorayunara/sorayunara
Cross-Compilation CLI:	<code>sora build --target <target-triple></code>
Status:	Official Production Specification (LTS)
License:	MIT Open Source License

1. Ubiquitous Platform Architecture Overview

Sorayunara ([.sora](#)) is engineered from the ground up to achieve **100% universal platform portability**. No matter what platform, hardware architecture, operating system, or execution environment a developer targets—from supercomputers to 8-bit microcontrollers, mobile smartphones, or browser sandboxes—Sorayunara provides first-class native compilation and runtime execution.

Universal Portability Guarantee: Any platform possessing an LLVM backend, an ANSI C99/C11 compiler (GCC, Clang, MSVC, SDCC), or a WebAssembly virtual machine can build, run, and execute Sorayunara code natively without behavioral divergence.

Multi-Target Compiler Pipeline



2. Tier 1: Production Desktop, Cloud & Server Platforms

Tier 1 targets are guaranteed to build, run, and pass 100% of automated CI test matrices on every commit. Pre-compiled binaries and official installers are distributed for all Tier 1 architectures.

Target Triple	Operating System	Architecture	Backend Engine	ABI / Linker	Tier
<code>x86_64-unknown-linux-gnu</code>	Linux (Ubuntu, Debian, Fedora, RHEL)	x86_64 (64-bit AMD/Intel)	LLVM Native	SysV ABI / GNU ld, lld	Tier 1
<code>x86_64-unknown-linux-musl</code>	Linux Static (Alpine, Containers)	x86_64	LLVM Native	SysV ABI / Static musl	Tier 1
<code>aarch64-unknown-linux-gnu</code>	Linux (AWS Graviton, Ampere, Pi 4/5)	ARM64 / AArch64	LLVM Native	AAPCS64 / lld	Tier 1
<code>aarch64-unknown-linux-musl</code>	Linux Static ARM (Embedded Containers)	ARM64 / AArch64	LLVM Native	AAPCS64 / Static musl	Tier 1
<code>x86_64-pc-windows-msvc</code>	Microsoft Windows 10, 11, Server	x86_64	LLVM Native	MSVC ABI / link.exe, lld-link	Tier 1
<code>aarch64-pc-windows-msvc</code>	Windows on ARM (Snapdragon X Elite)	ARM64	LLVM Native	MSVC ARM64 / lld-link	Tier 1
<code>aarch64-apple-darwin</code>	macOS (Apple Silicon M1, M2, M3, M4)	ARM64	LLVM Native	Darwin Mach-O / ld64	Tier 1
<code>x86_64-apple-darwin</code>	macOS (Intel Core i5/i7/i9/Xeon)	x86_64	LLVM Native	Darwin Mach-O / ld64	Tier 1
<code>wasm32-unknown-unknown</code>	Modern Web Browsers (Chrome, Safari, Edge)	WebAssembly 32-bit	WASM Backend	WASM Bytecode / wasm-ld	Tier 1
<code>wasm32-wasi</code>	Cloud Edge (Cloudflare, Deno, Wasmtime)	WebAssembly with WASI	WASM Backend	WASI POSIX / wasm-ld	Tier 1

3. Tier 2: Mobile & Unix Ecosystem Platforms

Tier 2 targets provide complete compiler support and automated cross-compilation toolchains for mobile apps, Unix derivatives, and next-generation RISC-V compute servers.

Target Triple	Platform / Ecosystem	Architecture	Toolchain & Linking Strategy	Tier
<code>aarch64-linux-android</code>	Android 8.0+ (ARM64 Native NDK)	ARM64	Android NDK / Clang toolchain / JNI Wrapper	Tier 2
<code>armv7-linux-androideabi</code>	Android Legacy (ARMv7 32-bit)	ARMv7-A	Android NDK / Thumb-2 mode	Tier 2
<code>aarch64-apple-ios</code>	iOS / iPadOS / watchOS Devices	ARM64	Xcode iOS SDK / Static C-ABI Framework	Tier 2
<code>x86_64-apple-ios</code>	iOS Simulator (macOS Intel)	x86_64	Xcode iOS Simulator SDK	Tier 2
<code>riscv64gc-unknown-linux-gnu</code>	RISC-V 64 Linux (SiFive, StarFive)	RISC-V 64 (RV64GC)	LLVM RISCV64 / GNU ABI	Tier 2
<code>x86_64-unknown-freebsd</code>	FreeBSD 13, 14	x86_64	LLVM Native / FreeBSD ELF	Tier 2
<code>x86_64-unknown-openbsd</code>	OpenBSD Secure OS	x86_64	LLVM Native / OpenBSD ELF pledge/unveil	Tier 2

4. Tier 3: Bare-Metal Microcontrollers, IoT & Real-Time OS

Sorayunara provides zero-overhead execution for embedded electronics, IoT microcontrollers, and Real-Time Operating Systems (FreeRTOS, Zephyr, RT-Thread) through its direct ANSI C99 transpiler and embedded LLVM emitter.

Target Triple / Hardware	Hardware Platform / Chipset	CPU Architecture	RAM / Flash Budget	Tier
<code>thumbv6m-none-eabi</code>	Raspberry Pi RP2040, STM32F0, SAMD21	ARM Cortex-M0 / M0+	≥ 16 KB RAM / 64 KB Flash	Tier 3
<code>thumbv7m-none-eabi</code>	STM32F1, STM32F3, NXP LPC1700	ARM Cortex-M3	≥ 32 KB RAM / 128 KB Flash	Tier 3
<code>thumbv7em-none-eabihf</code>	STM32F4, STM32F7, Teensy 4.0/4.1, nRF52840	ARM Cortex-M4F / M7F (Hardware FPU)	≥ 64 KB RAM / 256 KB Flash	Tier 3
<code>riscv32imc-unknown-none-elf</code>	Espressif ESP32-C3, GD32V, Kendryte K210	RISC-V 32 (RV32IMC)	≥ 32 KB RAM / 128 KB Flash	Tier 3
<code>xtensa-esp32-none-elf</code>	Espressif ESP32, ESP8266, ESP32-S3	Xtensa LX6 / LX7 Dual-Core	≥ 128 KB RAM / 512 KB Flash	Tier 3
<code>avr-unknown-none</code>	Arduino Uno, Nano, Mega 2560	8-bit AVR (ATmega328P / 2560)	≥ 2 KB RAM / 32 KB Flash	Tier 3
<code>c99-generic-embedded</code>	Any microcontroller with standard C compiler	Custom / Automotive / Avionics MCU	Arbitrary (Micro-runtime)	Tier 3

5. Tier 4: GPU, AI Accelerators & Heterogeneous Compute

For high-performance AI, linear algebra, tensor computation, and graphics, Sorayunara emits high-performance kernels and compute shaders.

Target Driver / Backend	Hardware Ecosystem	Output Format	Compute Model	Tier
<code>nvptx64-nvidia-cuda</code>	NVIDIA GPUs (GeForce, RTX, Tesla, A100, H100)	PTX Bytecode / CUDA C	Massively Parallel SIMT Kernels	Tier 4
<code>amdgcncn-amd-amdhsa</code>	AMD Radeon / Instinct (ROCm compute)	AMDGCN Machine Code / HIP	Heterogeneous Compute Kernels	Tier 4
<code>spirv-unknown-unknown</code>	Vulkan, OpenCL, Intel oneAPI GPUs	SPIR-V Binary Shader	Standard GPU Compute Shaders	Tier 4
<code>wgsl-webgpu</code>	Browser WebGPU Compute (Chrome, Safari)	WGSL Text Shaders	Web-accelerated Matrix Tensor Ops	Tier 4

6. CLI Cross-Compilation Workflows

6.1 Target Specification Commands

Developers can cross-compile any Sorayunara project with a single CLI parameter:

```
# 1. Compile for Linux x86_64 server
sora build --target x86_64-unknown-linux-gnu main.sora

# 2. Compile for Apple Silicon ARM64 (macOS)
sora build --target aarch64-apple-darwin main.sora

# 3. Compile for Windows x86_64 binary
sora build --target x86_64-pc-windows-msvc main.sora

# 4. Compile WebAssembly for Browser or Cloud Edge
sora build --target wasm32-wasi main.sora

# 5. Compile for Embedded Microcontroller (STM32 / RP2040)
sora build --target thumbv7em-none-eabihf --no-std main.sora

# 6. Transpile to Standalone Portable ANSI C99
sora build --target c99-generic-embedded main.sora
```

6.2 Standard Library Feature Availability Matrix

Module Category	Desktop / Server (Tier 1)	Mobile (Tier 2)	WebAssembly (Tier 1/2)	Embedded / Bare-Metal (Tier 3)
Core Math & Logic (<code>std.math</code> , <code>std.types</code>)	✔ Full Support	✔ Full Support	✔ Full Support	✔ Full Support (Zero-alloc)
Collections & Strings (<code>std.collections</code>)	✔ Full Support	✔ Full Support	✔ Full Support	✔ Static / Bump Allocator
File System & I/O (<code>std.fs</code> , <code>std.io</code>)	✔ Native OS Filesystem	✔ App Sandbox Storage	✔ Virtual WASI Filesystem	⚠ Flash ROM / LittleFS
Networking & HTTP (<code>std.net</code> , <code>std.http</code>)	✔ TCP/UDP/TLS Sockets	✔ Native Mobile Sockets	✔ Fetch API / WebSocket	⚠ LwIP / WiFi HAL (ESP32)
Concurrency & Actors (<code>std.channel</code>)	✔ Multi-Threaded Threads	✔ Multi-Threaded Coroutines	✔ Event-Loop Coroutines	⚠ FreeRTOS Task Scheduling
GPU & SIMD (<code>std.simd</code> , <code>std.gpu</code>)	✔ AVX-512 / NEON / CUDA	✔ ARM NEON / Metal	✔ WebAssembly SIMD128	✖ N/A

6.3 Summary of Compatibility

With this comprehensive 4-Tier Target Matrix, Sorayunara accomplishes universal execution versatility, allowing a single codebase written in `.sora` to seamlessly power high-throughput web backends, responsive mobile applications, interactive browser UIs, and mission-critical embedded IoT devices.